

ADO.NET

1. Introduction to ADO.NET

- **ADO.NET** is the core data access technology in .NET. It provides a set of classes to connect to databases, execute commands, and read results.
- It uses a **data provider** model:
 - Providers exist for SQL Server, OLE DB, ODBC, Oracle, etc.
 - For SQL Server, the modern provider is **Microsoft.Data.SqlClient** (NuGet package **Microsoft.Data.SqlClient**).
(In .NET Framework the legacy **System.Data.SqlClient** is still available, but **Microsoft.Data.SqlClient** is the recommended path for .NET Core / .NET 5+ and all new projects.)
- Core components common to most data providers:
 - **Connection** – establishes a session with the data source.
 - **Command** – executes T-SQL statements or stored procedures.
 - **DataReader** – fast, forward-only, read-only stream of rows (connected architecture).
 - **DataAdapter** – bridge between a dataset and the database (disconnected architecture).
- ADO.NET supports two working models:
 - **Connected architecture** – a live connection is maintained while data is being processed (using **DataReader**).
 - **Disconnected architecture** – data is retrieved and stored in in-memory structures (**DataSet/DataTable**) and the connection is released early.

2. SQL Server Overview

- **Microsoft SQL Server** is a relational database management system (RDBMS) that stores and retrieves data.
- Microsoft SQL Server Editions:
 - **Enterprise** – paid (expensive), large organizations/applications, maximum performance, HA features.
 - **Standard** – paid, small to medium medium, production workload.
 - **Developer** – free, full feature set, but cannot be used in production.
 - **Express** – free, limited resources, ideal for desktop/small applications.
 - **Express LocalDB** - lightweight dev edition for local dev & testing, installed in Visual Studio.
- **Instance**: a single installation of SQL Server. The default instance is reached by just the server name (e.g., **localhost**); named instances use the format **ServerName\InstanceName**.
- **Database (Initial Catalog)**: a container for tables, views, stored procedures, etc.
- **Authentication modes**:
 - **Windows Authentication** (Integrated Security) – uses the Windows account that runs the application. Recommended when both client and server are in the same Windows domain or on the same machine. SQL Server Express LocalDB only supports Windows Authentication.
 - **SQL Server Authentication** – uses a username and password created inside SQL Server (e.g., the **sa** account).
- Tools to interact with SQL Server:
 - **SQL Server Management Studio (SSMS)** – full-featured graphical tool.

- **Azure Data Studio** – lightweight, cross-platform alternative.
- **Visual Studio** – built-in tools like SQL Server Object Explorer.
- In ADO.NET, the provider talks to SQL Server via the **Tabular Data Stream (TDS)** protocol.

3. Connection Setup

- A **connection string** is a semicolon-delimited list of **key=value** pairs that specifies how to reach the database and how to authenticate.
- Essential keywords for **SQL Server**:
 - **Data Source** (or **Server**, **Addr**) – the server name or IP, optionally with instance name.
 - **Initial Catalog** (or **Database**) – the database name.
 - **Integrated Security** (or **Trusted_Connection**) – **true/SSPI** for Windows auth.
 - **User ID** and **Password** – used only with SQL Server Authentication.
- **Common connection string patterns**:
 - **Windows Authentication (LocalDB or local instance)**:

```
"Server=(localdb)\MSSQLLocalDB;Database=MyDB;Integrated Security=True;"
```
 - **SQL Server Authentication**:

```
"Server=192.168.1.100;Database=MyDB;User Id=myUser;Password=myPass;"
```
 - **Encrypt** – Starting with **Microsoft.Data.SqlClient** 4.0+, the default is **Encrypt=True**. When connecting to a local development server, you often add **TrustServerCertificate=True** to avoid certificate errors:

```
"Server=192.168.1.100,1433;Database=MyDB;Integrated Security=True;Encrypt=True;TrustServerCertificate=True;"
```
- **Connection pooling** is enabled by default. When you close a connection, it returns to a pool instead of truly disconnecting, greatly improving performance on re-opens.
- **Storing connection strings securely**:
 - **Never hard-code credentials** in source code.
 - Place the connection string in configuration files and read it at runtime:
 - **appsettings.json** (ASP.NET Core / modern .NET apps)
 - **App.config / Web.config** (.NET Framework)
 - Example using **appsettings.json**:

```
{
  "ConnectionStrings": {
    "Default": "Server=(localdb)\MSSQLLocalDB;Database=MyDB;Integrated Security=True;"
  }
}
```

Read it via `IConfiguration`:

```
string connStr = Configuration.GetConnectionString("Default");
```

4. Creating and Opening a Connection

- The `SqlConnection` class (namespace `Microsoft.Data.SqlClient`) represents a connection to SQL Server.
- Always wrap it in a `using` statement to guarantee the connection is closed and returned to the pool, even if exceptions occur.

```
using Microsoft.Data.SqlClient;

string connectionString = "Server=.;Database=MyDB;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True;";
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();
    Console.WriteLine("Connection opened successfully.");
    // use the connection...
} // connection is automatically closed and disposed here
```

- `Open()` throws an exception if the server is unreachable, the credentials are wrong, etc. Basic error handling can catch `SqlException`.

MSDN References

- [ADO.NET Overview](#)
- [Microsoft.Data.SqlClient documentation](#)
- [SqlConnection Class](#)
- [Connection Strings \(SqlClient\)](#)

5. Connected Architecture

- The **connected model** works with a live, open connection to the database while data is processed.
- Core components:
 - `SqlConnection` – the active database session.
 - `SqlCommand` – T-SQL statement or stored procedure to execute.
 - `SqlDataReader` – fast, forward-only, read-only stream of rows.
- **Key principle:** The connection must remain open while the `DataReader` is in use.
- **Typical workflow:**
 1. Create and open the `SqlConnection`.
 2. Create a `SqlCommand` (associating it with the connection).
 3. Execute the command with `ExecuteReader()` to get the `SqlDataReader`.
 4. Loop through rows with `reader.Read()` and access column values.

5. Close the reader and connection (automatic with `using` blocks).

```
string connStr = "Server=.;Database=MyDB;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True;";
using (SqlConnection connection = new SqlConnection(connStr)) {
    connection.Open();
    using (SqlCommand command = new SqlCommand("SELECT Id, Name FROM Products",
        connection)) {
        using (SqlDataReader reader = command.ExecuteReader()) {
            while (reader.Read()) {
                int id = reader.GetInt32(0);           // column index
                string name = reader.GetString(1);
                // or by column name: reader["Name"]
                Console.WriteLine($"{id}: {name}");
            }
        }
    }
} // connection closed here automatically
```

- **Accessing column values:** use indexer by column name or ordinal, or strongly-typed methods like `GetInt32`, `GetString`, `GetDateTime`, etc. Always check for `DBNull` if the column allows nulls.
- **Multiple result sets:** Use `reader.NextResult()` inside the loop to advance to the next result set if your command returns more than one.
- **Remember:** A `DataReader` holds the connection open. Never close the connection before you finish reading. The `using` blocks naturally handle this.
- For MySQL users: `MySqlDataReader` works identically; the pattern is the same.

6. Preventing SQL Injection

- **SQL injection** occurs when malicious user input is directly concatenated into a SQL statement, altering its logic.
- **Unsafe example:**

```
string userInput = "' OR 1=1";
string sql = "SELECT * FROM Products WHERE Name = '" + userInput + "'";
// The executed SQL becomes: SELECT * FROM Products WHERE Name = ' OR 1=1;
// Fetch all data
```

- **Defense: Parameterized queries**
 - Use `SqlParameter` to send user input as a parameter, never as part of the SQL text.
 - The provider sends the parameter value separately from the SQL, eliminating any possibility of SQL code being injected.
 - Parameter names use the `@` prefix.

```
string userInput = Console.ReadLine();
using (SqlConnection connection = new SqlConnection(connStr)) {
```

```

connection.Open();
string sql = "SELECT * FROM Products WHERE Name = @name";
using (SqlCommand command = new SqlCommand(sql, connection)) {
    // Add parameter
    command.Parameters.AddWithValue("@name", userInput);
    // Better: command.Parameters.Add("@name", SqlDbType.NVarChar, 50).Value =
userInput;
    using (SqlDataReader reader = command.ExecuteReader()) {
        // process results
    }
}
}

```

- **AddWithValue vs explicit Add:** `AddWithValue` infers the database type from the .NET value, which may cause performance issues (e.g., implicit conversions). For production, prefer `Add` with explicit `SqlDbType` and size.
- **Stored procedures are also safe** – as long as you never concatenate user input inside the procedure. Always parameterize.

7. Calling Stored Procedures

- Stored procedures are pre-compiled SQL code stored in the database. You call them by setting `CommandType` to `StoredProcedure`.
- Steps:
 1. Set `command.CommandType = CommandType.StoredProcedure`;
 2. Set `command.CommandText` to the procedure name.
 3. Add parameters matching the procedure's parameters.
 4. Execute (`ExecuteReader`, `ExecuteNonQuery`, or `ExecuteScalar`).
- **Input parameters:** Add with `Parameters.Add` or `AddWithValue`.
- **Output parameters:** Set `Direction = ParameterDirection.Output` and read value after execution.
- **Return value:** An integer return value is captured with a parameter of `Direction = ParameterDirection.ReturnValue` (usually first parameter).

```

using (SqlConnection connection = new SqlConnection(connStr)) {
    connection.Open();
    using (SqlCommand command = new SqlCommand("GetProductsByCategory",
connection)) {
        command.CommandType = CommandType.StoredProcedure;
        // Input parameter
        command.Parameters.Add(new SqlParameter("@CategoryId",
SqlDbType.Int)).Value = 5;
        // Output parameter
        SqlParameter outParam = new SqlParameter("@RowCount", SqlDbType.Int) {
            Direction = ParameterDirection.Output
        };
    }
}

```

```

        command.Parameters.Add(outParam);
        using (SqlDataReader reader = command.ExecuteReader()) {
            while (reader.Read()) {
                Console.WriteLine(reader["ProductName"]);
                // ...
            }
        }
        // Output parameter available after reader is closed
        int rowCount = (int)outParam.Value;
        Console.WriteLine($"Total rows: {rowCount}");
    }
}

```

- **ExecuteNonQuery()** for procedures that modify data but do not return rows. Output/return values are available immediately.
- For MySQL users: the pattern is identical with **MySqlCommand** , just the SQL Server specific syntax for output parameter handling is slightly different (MySQL uses **MySqlCommandType** etc.).

MSDN References

- [SqlDataReader Class](#)
- [SqlParameter Class](#)
- [CommandType Enum](#)
- [SQL Injection \(cheat sheet & prevention\)](#)
- [Stored Procedures in ADO.NET](#)

8. The DAO Class

- The **DAO pattern** isolates all database interaction code into dedicated classes (usually one per table or logical entity). This keeps your business logic clean and makes maintenance easier.
- The DAO a.k.a. DAL i.e. **Data Access Layer**.
- A typical DAO class for a **Product** entity might expose methods like:
 - **GetAll()** → **List<Product>**
 - **GetById(int id)** → **Product**
 - **Insert(Product product)** → **int** (new ID)
 - **Update(Product product)** → **bool** (success)
 - **Delete(int id)** → **bool**
- **Structure best practices:**
 - Keep connection string configuration centralised (e.g., retrieved via dependency injection or a static helper).
 - Always use **using** blocks for **SqlConnection**, **SqlCommand**, **SqlDataReader**.
 - Always use **parameterised queries** (never concatenate input).
 - Return domain objects (e.g., **Product**) rather than raw **DataReader** or **DataTable** – this keeps the rest of the application database-agnostic.

```

public class ProductDao {
    private readonly string _connectionString;
    public ProductDao(string connectionString) => _connectionString =

```

```
connectionString;

public List<Product> GetAll() {
    var products = new List<Product>();
    string sql = "SELECT Id, Name, Price FROM Products";
    using (var conn = new SqlConnection(_connectionString)) {
        conn.Open();
        using (var cmd = new SqlCommand(sql, conn)) {
            using (var reader = cmd.ExecuteReader()) {
                while (reader.Read()) {
                    products.Add(new Product {
                        Id = reader.GetInt32(0),
                        Name = reader.GetString(1),
                        Price = reader.GetDecimal(2)
                    });
                }
            }
        }
    }
    return products;
}

public Product GetById(int id) {
    string sql = "SELECT Id, Name, Price FROM Products WHERE Id = @id";
    using (var conn = new SqlConnection(_connectionString)) {
        conn.Open();
        using (var cmd = new SqlCommand(sql, conn)) {
            cmd.Parameters.Add("@id", SqlDbType.Int).Value = id;
            using (var reader = cmd.ExecuteReader()) {
                if (reader.Read()) {
                    return new Product {
                        Id = reader.GetInt32(0),
                        Name = reader.GetString(1),
                        Price = reader.GetDecimal(2)
                    };
                }
                else
                    return null;
            }
        }
    }
}

public int Insert(Product product) {
    string sql = @"INSERT INTO Products (Name, Price) VALUES (@name, @price);
                SELECT SCOPE_IDENTITY();"; // returns the new Id
    using (var conn = new SqlConnection(_connectionString)) {
        conn.Open();
        using (var cmd = new SqlCommand(sql, conn)) {
            cmd.Parameters.Add("@name", SqlDbType.NVarChar, 100).Value =
product.Name;
            cmd.Parameters.Add("@price", SqlDbType.Decimal).Value =
product.Price;
            return Convert.ToInt32(cmd.ExecuteScalar());
        }
    }
}
```

```

    }
}

public bool Update(Product product) {
    string sql = "UPDATE Products SET Name = @name, Price = @price WHERE Id = @id";
    using (var conn = new SqlConnection(_connectionString)) {
        conn.Open();
        using (var cmd = new SqlCommand(sql, conn))
        {
            cmd.Parameters.Add("@name", SqlDbType.NVarChar, 100).Value =
product.Name;
            cmd.Parameters.Add("@price", SqlDbType.Decimal).Value =
product.Price;
            cmd.Parameters.Add("@id", SqlDbType.Int).Value = product.Id;
            int rowsAffected = cmd.ExecuteNonQuery();
            return rowsAffected > 0;
        }
    }
}

public bool Delete(int id)
{
    string sql = "DELETE FROM Products WHERE Id = @id";
    using (var conn = new SqlConnection(_connectionString)) {
        conn.Open();
        using (var cmd = new SqlCommand(sql, conn)) {
            cmd.Parameters.Add("@id", SqlDbType.Int).Value = id;
            int rows = cmd.ExecuteNonQuery();
            return rows > 0;
        }
    }
}
}

```

- **SCOPE_IDENTITY()** returns the last identity value generated in the current scope – safer than @@IDENTITY when triggers are involved.

9. Transaction Management

- A **transaction** groups multiple database operations into a single unit of work that either succeeds completely (**COMMIT**) or fails completely (**ROLLBACK**), preserving database consistency (ACID properties).
- In ADO.NET, use **SqlTransaction** obtained from the connection before executing commands.
- **Steps:**
 1. Open the connection.
 2. Call **connection.BeginTransaction()** (optionally specify isolation level).
 3. Assign the transaction to each **SqlCommand** via **command.Transaction**.
 4. Execute commands.
 5. If everything succeeds, call **transaction.Commit()**.

6. If an exception occurs, call `transaction.Rollback()` in a `catch` block.
7. Dispose the connection and transaction (a `using` block on the transaction is highly recommended).

```
using (var conn = new SqlConnection(_connectionString)) {
    conn.Open();
    // Begin transaction
    using (SqlTransaction transaction = conn.BeginTransaction()) {
        try {
            // First operation
            string sql1 = "UPDATE Accounts SET Balance = Balance - 100 WHERE Id =
@from";

            using (var cmd1 = new SqlCommand(sql1, conn, transaction)) {
                cmd1.Parameters.AddWithValue("@from", 1);
                cmd1.ExecuteNonQuery();
            }
            // Second operation
            string sql2 = "UPDATE Accounts SET Balance = Balance + 100 WHERE Id =
@to";

            using (var cmd2 = new SqlCommand(sql2, conn, transaction)) {
                cmd2.Parameters.AddWithValue("@to", 2);
                cmd2.ExecuteNonQuery();
            }
            // Everything worked - commit
            transaction.Commit();
        }
        catch {
            // Something went wrong - roll back
            transaction.Rollback();
            throw; // re-throw if the caller needs to know
        }
    }
} // connection closed automatically
```

- The `using` block on `SqlTransaction` ensures that if `Commit` or `Rollback` is not called explicitly, a rollback is performed on dispose.
- **Isolation levels** control locking and read phenomena. Common levels: `ReadCommitted` (default), `RepeatableRead`, `Serializable`, `ReadUncommitted`. Specify in `BeginTransaction(isolationLevel)`.

10. ADO.NET Disconnected Architecture

- The **disconnected model** retrieves data into in-memory objects (`DataSet` / `DataTable`) and immediately releases the database connection. You then work offline and later push changes back in a single batch.
- Key classes:
 - **DataSet** – an in-memory cache of data that can contain multiple `DataTable` objects and relationships.
 - **DataTable** – represents one table of data.

- **SqlDataAdapter** – the bridge: it uses commands (**SelectCommand**, **InsertCommand**, **UpdateCommand**, **DeleteCommand**) to fill data and later update the database.

- **Typical workflow:**

1. Create a **SqlConnection** and **SqlDataAdapter**.
2. Use **adapter.Fill(dataTable)** or **adapter.Fill(dataSet, "TableName")** – this opens the connection, executes the select, fills the table, and closes the connection.
3. Work with the **DataTable** in memory: iterate, add rows, modify, delete.
4. Call **adapter.Update(dataTable)** to push all changes (inserts, updates, deletes) back to the database. The adapter uses the appropriate INSERT/UPDATE/DELETE commands (or a **CommandBuilder** to auto-generate them).

```

DataTable productsTable = new DataTable("Products");
string selectSql = "SELECT Id, Name, Price FROM Products";
using (var conn = new SqlConnection(_connectionString)) {
    using (var adapter = new SqlDataAdapter(selectSql, conn)) {
        // Auto-generate insert/update/delete commands
        using (var builder = new SqlCommandBuilder(adapter)) {
            // Fill the table (opens and closes connection automatically)
            adapter.Fill(productsTable);
            // Work offline
            foreach (DataRow row in productsTable.Rows)
                Console.WriteLine($"{row["Id"]}: {row["Name"]}");

            // Add a new row
            DataRow newRow = productsTable.NewRow();
            newRow["Name"] = "New Product";
            newRow["Price"] = 29.99M;
            productsTable.Rows.Add(newRow);

            // Update a row
            DataRow firstRow = productsTable.Rows[0];
            firstRow["Price"] = 19.99M;

            // Delete a row
            productsTable.Rows[1].Delete();

            // Push all changes back to the database
            int rowsUpdated = adapter.Update(productsTable);
            Console.WriteLine($"Rows affected: {rowsUpdated}");
        }
    }
}

```

- **SqlCommandBuilder** automatically generates the **InsertCommand**, **UpdateCommand**, and **DeleteCommand** by analysing the **SelectCommand**. It only works for simple single-table queries and requires a primary key in the result set.
- For **complex scenarios**, you provide the insert/update/delete commands manually (as stored procedures or parameterised SQL).

- The `DataTable` tracks row states (`Added`, `Modified`, `Deleted`) and `AcceptChanges()` resets them after a successful update.
- **Advantage:** You hold the connection for a very short time – ideal for web apps and desktop apps with large datasets.

MSDN References

- [Designing Data Access Objects \(DAO\) in .NET](#)
- [SqlTransaction Class](#)
- [Transaction Management in ADO.NET](#)
- [DataSet, DataTable, DataAdapter](#)
- [SqlDataAdapter Class](#)
- [SqlCommandBuilder Class](#)

Expression Trees

1. Introduction

- An **expression tree** is a data structure that represents code in a tree format, where each node is an expression (e.g., a constant, parameter, method call, or binary operation).
- Unlike a compiled delegate, an expression tree can be **analysed, modified, or translated** at runtime before execution.
- They are fundamental to **LINQ providers** (like Entity Framework Core) that convert C# query expressions into SQL.
- When you assign a lambda to a variable of type `Expression<TDelegate>`, the compiler builds an expression tree instead of IL.
- Expressions reside in the `System.Linq.Expressions` namespace.

2. Core Concepts

- `Expression<TDelegate>`
 - Represents a strongly-typed lambda expression as a tree.
 - Example: `Expression<Func<int, int, int>> expr = (x, y) => x + y;` stores the addition operation as an expression tree.
- **Node types** (all derive from `Expression`):
 - `ConstantExpression` – a literal value (e.g., `5`).
 - `ParameterExpression` – a named parameter (e.g., `x`).
 - `BinaryExpression` – an operation with two operands (e.g., `+`, `==`, `&&`).
 - `MethodCallExpression` – a call to a method (e.g., `Console.WriteLine`).
 - `LambdaExpression` – a lambda expression itself, containing a body and parameters.
 - Many more (`UnaryExpression`, `MemberExpression`, `ConditionalExpression`, etc.).
- **Immutability:** Expression trees are immutable. Altering a tree means creating a new one.
- **Expression vs. Delegate**

- `Func<int, int, int> func = (x, y) => x + y;` → compiled into IL immediately, can be invoked.
- `Expression<Func<int, int, int>> expr = (x, y) => x + y;` → stored as a tree that can be inspected, compiled later with `expr.Compile()`.

3. Creating Expression Trees via Lambda

- The compiler automatically converts a lambda expression to an expression tree when the target type is `Expression<T>`.

```
Expression<Func<int, int, int>> addition = (a, b) => a + b;

// Inspect the tree
Console.WriteLine(addition.Body);    // (a + b)
Console.WriteLine(addition.Parameters[0].Name); // a
```

- This is the simplest way to get an expression tree when the expression is known at compile time.

4. Building Expression Trees Manually

- Use static factory methods of the `Expression` class to build a tree programmatically.

```
using System.Linq.Expressions;

// Create parameters
ParameterExpression paramA = Expression.Parameter(typeof(int), "a");
ParameterExpression paramB = Expression.Parameter(typeof(int), "b");

// Build body: a * b
BinaryExpression body = Expression.Multiply(paramA, paramB);

// Create lambda: (a, b) => a * b
Expression<Func<int, int, int>> multiplyExpr
    = Expression.Lambda<Func<int, int, int>>(body, paramA, paramB);

// Compile and execute
Func<int, int, int> multiply = multiplyExpr.Compile();
int result = multiply(6, 7);    // 42
```

- Key `Expression` methods:
 - `Expression.Parameter` – define parameters.
 - `Expression.Constant` – wrap a constant value.
 - `Expression.Add`, `Subtract`, `Multiply`, `Divide` – arithmetic.
 - `Expression.Equal`, `NotEqual`, `GreaterThan`, etc. – comparisons.
 - `Expression.AndAlso`, `OrElse` – logical short-circuit operators (`&&`, `||`).
 - `Expression.Call` – method calls.
 - `Expression.Property` or `Expression.Field` – member access.

- `Expression.Lambda<TDelegate>` – assemble into a lambda expression.

5. Compiling and Executing Expression Trees

- `Expression<TDelegate>.Compile()` converts the tree into an executable delegate of type `TDelegate`.

```
Expression<Func<int, bool>> isPositive = num => num > 0;
Func<int, bool> compiled = isPositive.Compile();
Console.WriteLine(compiled(5)); // True
```

- **Caveats:**
 - Compilation is relatively expensive – compile once and cache the delegate if used repeatedly.
 - In LINQ providers (like EF Core), the expression tree is never compiled; it is translated into SQL instead.

6. Inspecting and Traversing Expression Trees

- `ExpressionVisitor` (abstract base class) allows you to walk every node in the tree.
- Override `Visit` methods to inspect or modify nodes.
- **Example: a simple visitor that prints all binary operations**

```
class PrintingVisitor : ExpressionVisitor {
    protected override Expression VisitBinary(BinaryExpression node) {
        Console.WriteLine($"Binary: {node.NodeType} with operands " + $"
{node.Left} and {node.Right}");
        return base.VisitBinary(node);
    }
}

// Usage
Expression<Func<int, int, int>> expr = (x, y) => (x * 2) + y;
var visitor = new PrintingVisitor();
visitor.Visit(expr);
// Output:
// Binary: Add with operands (x * 2) and y
// Binary: Multiply with operands x and 2
```

- The visitor pattern is used extensively for translation (like converting a .NET expression tree into SQL internally by EF core).

7. Common Use Cases

- **Dynamic Query Building**
Construct `IQueryable.Where()` predicates at runtime based on user input (e.g., filters in a search form).

- **LINQ Providers / ORMs**

Entity Framework Core, LINQ to SQL, and other providers take an expression tree and translate it into a domain-specific language (SQL, GraphQL, etc.).

- **Rule Engines**

Store business rules as expression trees that can be compiled or interpreted later.

- **Metaprogramming / Code Generation**

Create code dynamically at runtime, for example, writing a lightweight reflection substitute or a fast object mapper.

8. Sample: Building a Dynamic Filter Predicate

- Goal: create the expression `x => x.Age > 18` at runtime.

```
public class Person {  
    public string Name { get; set; }  
    public int Age { get; set; }  
}  
  
// Build the predicate  
ParameterExpression param = Expression.Parameter(typeof(Person), "x");  
// x.Age  
MemberExpression ageProperty = Expression.Property(param, "Age");  
// 18  
ConstantExpression ageValue = Expression.Constant(18);  
// x.Age > 18  
BinaryExpression ageCondition = Expression.GreaterThan(ageProperty, ageValue);  
  
// Build lambda  
Expression<Func<Person, bool>> filter = Expression.Lambda<Func<Person, bool>>  
(ageCondition, param);  
  
// Use it with IQueryable  
var people = new List<Person>().AsQueryable();  
var result = people.Where(filter);
```

- The constructed expression is fully usable with any LINQ provider that understands expression trees.

MSDN References

- [Expression Trees \(C# Programming Guide\)](#)
- [System.Linq.Expressions Namespace](#)
- [Expression Class](#)
- [ExpressionVisitor Class](#)
- [How to use expression trees to build dynamic queries \(C#\)](#)